

Programming Assignment: Unit 3

Hset Paing Htoo

Introduction to Computer Science Program, University of the People

CS 1102-01 Programming 1 AY2026-T5

Instructor : Sirajo Musa (Instructor)

July 6, 2026

Student Record Management System

1. Introduction

This assignment implements a console-based Student Record Management System in Java for a university administrator. The program demonstrates encapsulation through private instance variables, class-level (static) state and behavior through private static variables and static methods, and defensive error handling for invalid or missing input. The system is organized into three classes: `Student`, which models an individual student record; `StudentManagement`, which maintains the shared list of all students and exposes static operations to add, update, and retrieve them; and `Main`, which drives a menu-based administrator interface.

2. Concepts Applied

2.1 Encapsulation with Private Instance Variables

The `Student` class stores name, id, age, and grade as private instance variables. Each instance variable belongs to a specific `Student` object, so every student created from the class has its own independent copy of these fields (Eck, 2022, Section 4.2). Public getter and setter methods control how these fields are read and modified from outside the class, which protects the internal state of each `Student` object from uncontrolled changes.

2.2 Static Variables and Static Methods

The `StudentManagement` class uses private static variables — a `List<Student>` to hold every student record and an `int` counter for the total number of students — because this information describes the system as a whole rather than any single student. According to Eck (2022, Section 4.2), a static variable belongs to the class itself, so only one copy exists no matter how many `Student` objects are created. All of `StudentManagement`'s methods (`addStudent`, `updateStudent`,

`viewStudent`, `viewAllStudents`) are declared static so that the administrator interface can call them directly on the class (e.g., `StudentManagement.addStudent(...)`) without needing to create a `StudentManagement` object.

2.3 Menu-Driven Administrator Interface

The `Main` class presents a numbered menu in a while loop, reads the administrator's choice, and uses a switch statement to dispatch to the correct operation. This mirrors the subroutine-based program structure described by Eck (2022, Section 4.3), where the main method coordinates smaller, single-purpose methods rather than containing all logic itself.

2.4 Error Handling

Two categories of invalid input are handled. First, non-numeric input for menu choices, student IDs, or ages is caught with a try-catch block around `Integer.parseInt()`, which throws a `NumberFormatException` on bad input (Eck, 2022, Section 3.7, as applied earlier in this course). Second, logical errors — such as looking up or updating a student ID that does not exist in the system — are handled explicitly by checking the result of a private helper method, `findStudentById()`, and printing a clear error message instead of allowing the program to crash or fail silently.

3. Program Code

The system consists of three files: Student.java, StudentManagement.java, and Main.java.

3.1 Student.java

```
public class Student {
    private String name;
    private int id;
    private int age;
    private String grade;

    public Student(String name, int id, int age, String grade) {
        this.name = name;
        this.id = id;
        this.age = age;
        this.grade = grade;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getId() { return id; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }

    @Override
    public String toString() {
        return "ID: " + id + " | Name: " + name
            + " | Age: " + age + " | Grade: " + grade;
    }
}
```

3.2 StudentManagement.java

```
import java.util.ArrayList;
import java.util.List;

public class StudentManagement {
    // Shared across the whole system - not tied to one Student
    private static List<Student> students = new ArrayList<>();
    private static int totalStudents = 0;

    public static void addStudent(String name, int id, int age, String
grade) {
        if (findStudentById(id) != null) {
            System.out.println("[Error] A student with ID " + id + "
already exists.");
            return;
        }
        Student newStudent = new Student(name, id, age, grade);
```

```

        students.add(newStudent);
        totalStudents++;
        System.out.println("[Success] Student added. Total students: " +
totalStudents);
    }

    public static void updateStudent(int id, String name, int age, String
grade) {
        Student student = findStudentById(id);
        if (student == null) {
            System.out.println("[Error] Student ID " + id + " not
found.");
            return;
        }
        if(!name.isEmpty()) {student.setName(name);}
        if(age > 0) {student.setAge(age);}
        if(!grade.isEmpty()) {student.setGrade(grade);}
        System.out.println("[Success] Student ID " + id + "was updated
successfully!");}

    public static void viewStudent(int id) {
        Student student = findStudentById(id);
        if (student == null) {
            System.out.println("[Error] Student ID " + id + " not
found.");
            return;
        }
        System.out.println(student);
    }

    public static void viewAllStudents() {
        if (students.isEmpty()) {
            System.out.println("[Info] No students in the system yet.");
            return;
        }
        for (Student s : students) {
            System.out.println(s);
        }
    }

    private static Student findStudentById(int id) {
        for (Student s : students) {
            if (s.getId() == id) {
                return s;
            }
        }
        return null;
    }

    public static int getTotalStudents() {
        return totalStudents;
    }
}

```

3.3 Main.java

```

import java.util.Scanner;

```

```

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean running = true;

        System.out.println("=== Student Record Management System ===");

        while (running) {
            printMenu();
            int choice = -1;
            try {
                System.out.print("Enter your choice: ");
                choice = Integer.parseInt(scanner.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("[Error] Invalid input. Enter a number
1-5.");
                continue;
            }

            switch (choice) {
                case 1: addStudentFlow(scanner); break;
                case 2: updateStudentFlow(scanner); break;
                case 3: viewStudentFlow(scanner); break;
                case 4: StudentManagement.viewAllStudents(); break;
                case 5:
                    System.out.println("Goodbye!");
                    running = false;
                    break;
                default:
                    System.out.println("[Error] Choose a number between 1
and 5.");
            }
        }
        scanner.close();
    }

    private static void printMenu() {
        System.out.println("\n--- Administrator Menu ---");
        System.out.println("1. Add New Student");
        System.out.println("2. Update Student Information");
        System.out.println("3. View Student Details");
        System.out.println("4. View All Students");
        System.out.println("5. Exit");
    }

    private static void addStudentFlow(Scanner scanner) {
        try {
            System.out.print("Enter student name: ");
            String name = scanner.nextLine().trim();
            if (name.isEmpty()) {
                System.out.println("[Error] Name cannot be empty.");
                return;
            }

            System.out.print("Enter student ID: ");
            int id = Integer.parseInt(scanner.nextLine().trim());

```

```

        System.out.print("Enter student age: ");
        int age = Integer.parseInt(scanner.nextLine().trim());

        System.out.print("Enter student grade: ");
        String grade = scanner.nextLine().trim();

        StudentManagement.addStudent(name, id, age, grade);
    } catch (NumberFormatException e) {
        System.out.println("[Error] ID and age must be numbers.");
    }
}

private static void updateStudentFlow(Scanner scanner) {
    try {
        System.out.print("Enter student ID to update: ");
        int id = Integer.parseInt(scanner.nextLine().trim());

        System.out.print("Enter new name: ");
        String name = scanner.nextLine().trim();

        System.out.print("Enter new age: ");
        int age = Integer.parseInt(scanner.nextLine().trim());

        System.out.print("Enter new grade: ");
        String grade = scanner.nextLine().trim();

        StudentManagement.updateStudent(id, name, age, grade);
    } catch (NumberFormatException e) {
        System.out.println("[Error] ID and age must be numbers.");
    }
}

private static void viewStudentFlow(Scanner scanner) {
    try {
        System.out.print("Enter student ID to view: ");
        int id = Integer.parseInt(scanner.nextLine().trim());
        StudentManagement.viewStudent(id);
    } catch (NumberFormatException e) {
        System.out.println("[Error] ID must be a number.");
    }
}
}

```

4. Code Explanation

4.1 Student Class

Student represents a single student record. Its four fields are private, so no other class can read or change them directly — access is only possible through the public getter and setter methods.

This is standard encapsulation: it keeps each Student object responsible for protecting its own data, and it means validation logic could later be added inside a setter (for example, rejecting a negative age) without changing any other class.

4.2 StudentManagement Class

StudentManagement holds the entire system's data in two private static variables: a `List<Student>` and an int total counter. Because these are static, they exist once for the whole class rather than once per object — there is never more than one master list of students, regardless of how many times the class is referenced. All public methods are static as well, so the Main class calls them directly (e.g., `StudentManagement.viewAllStudents()`) with no need to instantiate a StudentManagement object. The private helper method `findStudentById()` is used internally by `addStudent`, `updateStudent`, and `viewStudent` to search the list by ID; it returns null when no match is found, which each caller checks before proceeding, giving a single consistent place where the 'ID not found' error condition is detected.

4.3 Main Class and the Administrator Interface

`Main.main()` prints a welcome message and enters a while loop that keeps the menu running until the administrator chooses to exit (option 5). Each iteration prints the menu, reads the choice, and uses a switch statement to route to the matching private method: `addStudentFlow()`, `updateStudentFlow()`, or `viewStudentFlow()`. Each flow method is responsible only for collecting

and validating console input before handing the actual data operation off to StudentManagement — keeping the input-handling logic and the data-management logic cleanly separated.

4.4 Error Handling Strategy

Two kinds of errors are handled everywhere user input is involved. Invalid numeric input (letters typed where a number is expected) triggers a `NumberFormatException`, which is caught so the program prints a message and returns to the menu instead of crashing. Logical errors — such as updating or viewing a student ID that was never added — are caught by checking whether `findStudentById()` returned null, and printing '[Error] Student ID ... not found.' in that case.

Together these two checks mean the administrator interface never terminates unexpectedly due to bad input.

5. Program Output

The following screenshots demonstrate the program running: adding a new student, viewing student details, updating a student's information, attempting to view a non-existent student ID (error handling), and viewing all students.

```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-3
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-3$ javac Main.java
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-3$ ls
Main.class Main.java Student.class Student.java StudentManagement.class StudentManagement.java
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-3$ java Main
=== Student Record Management System ===

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
5. Exit
Enter your choice: 1
Enter student name: Aung Aung
Enter student ID: 101
Enter student age: 21
Enter student grade: A
[Info] Student added successfully. Total students: 1

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
```

```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-3
Enter your choice: 1
Enter student name: Su Su
Enter student ID: 102
Enter student age: 22
Enter student grade: B
[Info] Student added successfully. Total students: 2

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
5. Exit
Enter your choice: 3
Enter student ID to view: 101
ID: 101 | Name: Aung Aung | Age: 21 | Grade: A

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
5. Exit
Enter your choice:
```

```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-3

ID: 101 | Name: Aung Aung | Age: 22 | Grade: A+

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
5. Exit
Enter your choice: 2
Enter student ID to update: 101
Enter new name: Aung Aung
Enter new age: 22
Enter new grade: A+
[Info] Student with ID: 101 was updated successfully!

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
5. Exit
Enter your choice: 3
Enter student ID to view: 999
[Error] Student with ID: 999not found!
```

```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-3

5. Exit
Enter your choice: 3
Enter student ID to view: 999
[Error] Student with ID: 999not found!

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
5. Exit
Enter your choice: 4
ID: 101 | Name: Aung Aung | Age: 22 | Grade: A+
ID: 102 | Name: Su Su | Age: 22 | Grade: B

--- Administrator Menu ---
1. Add New Student
2. Update Student Information
3. View Student Details
4. View All Students
5. Exit
Enter your choice: 5
Goodbye!
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-3$ |
```

6. Instructions for Running the Program

- Save the three classes as Student.java, StudentManagement.java, and Main.java in the same folder.
- Open a terminal in that folder.
- Compile all files: `javac Student.java StudentManagement.java Main.java`
- Run the program: `java Main`
- Follow the on-screen menu (1-5) to add, update, view a single student, view all students, or exit.
- When adding or updating a student, enter the ID and age as whole numbers; the program will reject non-numeric input and re-show the menu.

7. Discussion

7.1 Design Decisions

Keeping the student list and total counter as private static fields inside StudentManagement, rather than as public fields or as fields on Main, follows the requirement that class-wide data be encapsulated and only reachable through the class's own static methods. Separating the three classes by responsibility — data (Student), storage and business logic (StudentManagement), and user interaction (Main) — also makes the system easier to extend later, for example by swapping the console interface for a graphical one without touching StudentManagement at all.

7.2 Limitations and Possible Improvements

- No persistence: all student records are lost when the program exits; a file- or database-backed store would fix this.
- No input validation beyond type checking: for example, a negative age or an empty grade string is currently accepted.
- Linear search: `findStudentById()` scans the whole list, which is fine for small class sizes but would not scale to a very large student body without an index such as a `HashMap` keyed by ID.
- No delete option: the current menu cannot remove a student record, which a full system would likely need.

8. Conclusion

This assignment applied static variables and static methods to build a shared, class-wide store of student records, alongside private instance variables to protect the data of each individual Student. The resulting Student Record Management System offers a menu-driven administrator interface backed by defensive error handling, satisfying the assignment's requirements for student updates, student management, an administrator interface, and robust handling of invalid or missing input.

References

Eck, D. J. (2022). Introduction to programming using Java, version 9, JavaFX edition.

<https://math.hws.edu/javanotes/>

Neso Academy. (2020, June 23). Static variables and static methods in Java [Video]. YouTube.

<https://youtu.be/3Ltlk7wFGi0>